

Conversations

Create and manage conversations to store and retrieve conversation state across Response API calls.

Create a conversation

POST <https://api.openai.com/v1/conversations>

Create a conversation.

Request body

items array Optional

Initial items to include in the conversation context. You may add up to 20 items at a time.

> Show possible types

metadata object or null Optional

Set of 16 key-value pairs that can be attached to an object. This can be useful for storing additional information about the object in a structured format, and querying for objects via API or the dashboard. Keys are strings with a maximum length of 64 characters. Values are strings with a maximum length of 512 characters.

Returns

Returns a [Conversation](#) object.

Example request

javascript 

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const conversation = await client.conversations.create({
5   metadata: { topic: "demo" },
6   items: [
7     { type: "message", role: "user", content: "Hello!" }
8   ],
9 });
10 console.log(conversation);
```

Response

```
1 {  
2   "id": "conv_123",  
3   "object": "conversation",  
4   "created_at": 1741900000,  
5   "metadata": {"topic": "demo"}  
6 }
```

Retrieve a conversation

GET https://api.openai.com/v1/conversations/{conversation_id}

Get a conversation

Path parameters

conversation_id string **Required**

The ID of the conversation to retrieve.

Returns

Returns a [Conversation](#) object.

Example request

javascript

```
1 import OpenAI from "openai";  
2 const client = new OpenAI();  
3  
4 const conversation = await client.conversations.retrieve("conv_123");  
5 console.log(conversation);
```

Response

```
1 {  
2   "id": "conv_123",  
3   "object": "conversation",  
4   "created_at": 1741900000,  
5   "metadata": {"topic": "demo"}  
6 }
```

Update a conversation

POST https://api.openai.com/v1/conversations/{conversation_id}

Update a conversation

Path parameters

conversation_id string **Required**

The ID of the conversation to update.

Request body

metadata map **Required**

Set of 16 key-value pairs that can be attached to an object. This can be useful for storing additional information about the object in a structured format, and querying for objects via API or the dashboard.

Keys are strings with a maximum length of 64 characters. Values are strings with a maximum length of 512 characters.

Returns

Returns the updated Conversation object.

Example request

javascript

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const updated = await client.conversations.update(
5   "conv_123",
6   { metadata: { topic: "project-x" } }
7 );
8 console.log(updated);
```

Response

```
1 {
2   "id": "conv_123",
3   "object": "conversation",
4   "created_at": 1741900000,
```

```
5  "metadata": {"topic": "project-x"}
6  }
```

Delete a conversation

```
DELETE https://api.openai.com/v1/conversations/{conversation_id}
```

Delete a conversation. Items in the conversation will not be deleted.

Path parameters

conversation_id string **Required**

The ID of the conversation to delete.

Returns

A success message.

Example request

javascript

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const deleted = await client.conversations.delete("conv_123");
5 console.log(deleted);
```

Response

```
1 {
2   "id": "conv_123",
3   "object": "conversation.deleted",
4   "deleted": true
5 }
```

List items

```
GET https://api.openai.com/v1/conversations/{conversation_id}/items
```

List all items for a conversation with the given ID.

Path parameters

conversation_id string **Required**

The ID of the conversation to list items for.

Query parameters

after string Optional

An item ID to list items after, used in pagination.

include array Optional

Specify additional output data to include in the model response. Currently supported values are:

- `web_search_call.action.sources` : Include the sources of the web search tool call.
- `code_interpreter_call.outputs` : Includes the outputs of python code execution in code interpreter tool call items.
- `computer_call_output.output.image_url` : Include image urls from the computer call output.
- `file_search_call.results` : Include the search results of the file search tool call.
- `message.input_image.image_url` : Include image urls from the input message.
- `message.output_text.logprobs` : Include logprobs with assistant messages.
- `reasoning.encrypted_content` : Includes an encrypted version of reasoning tokens in reasoning item outputs. This enables reasoning items to be used in multi-turn conversations when using the Responses API statelessly (like when the `store` parameter is set to `false` , or when an organization is enrolled in the zero data retention program).

limit integer Optional Defaults to 20

A limit on the number of objects to be returned. Limit can range between 1 and 100, and the default is 20.

order string Optional

The order to return the input items in. Default is `desc` .

- `asc` : Return the input items in ascending order.
- `desc` : Return the input items in descending order.

Returns

Returns a [list object](#) containing Conversation items.

Example request

javascript

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const items = await client.conversations.items.list("conv_123", { limit: 10 });
5 console.log(items.data);
```

Response

```
1 {
2   "object": "list",
3   "data": [
4     {
5       "type": "message",
6       "id": "msg_abc",
7       "status": "completed",
8       "role": "user",
9       "content": [
10        { "type": "input_text", "text": "Hello!" }
11      ]
12    }
13  ],
14  "first_id": "msg_abc",
15  "last_id": "msg_abc",
16  "has_more": false
17 }
```

Create items

POST https://api.openai.com/v1/conversations/{conversation_id}/items

Create items in a conversation with the given ID.

Path parameters

conversation_id string **Required**

The ID of the conversation to add the item to.

Query parameters

include array **Optional**

Additional fields to include in the response. See the `include` parameter for

Additional fields to include in the response. See the `include` parameter for listing [Conversation items above](#) for more information.

Request body

items array **Required**

The items to add to the conversation. You may add up to 20 items at a time.

Show possible types

Returns

Returns the list of added [items](#).

Example request

javascript

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const items = await client.conversations.items.create(
5   "conv_123",
6   {
7     items: [
8       {
9         type: "message",
10        role: "user",
11        content: [{ type: "input_text", text: "Hello!" }],
12      },
13      {
14        type: "message",
15        role: "user",
16        content: [{ type: "input_text", text: "How are you?" }],
17      },
18    ],
19  }
20 );
21 console.log(items.data);
```

Response

```
1 {
2   "object": "list",
3   "data": [
4     {
5       "type": "message",
6       "id": "msg_abc",
7       "status": "completed",
```

```
8     "role": "user",
9     "content": [
10        {"type": "input_text", "text": "Hello!"}
11    ]
12 },
13 {
14     "type": "message",
15     "id": "msg_def",
16     "status": "completed",
17     "role": "user",
18     "content": [
19        {"type": "input_text", "text": "How are you?"}
20    ]
21 }
22 ],
23 "first_id": "msg_abc",
24 "last_id": "msg_def",
25 "has_more": false
26 }
```

Retrieve an item

```
GET https://api.openai.com/v1/conversations/{conversation_id}/items/{item_id}
```

Get a single item from a conversation with the given IDs.

Path parameters

conversation_id string **Required**

The ID of the conversation that contains the item.

item_id string **Required**

The ID of the item to retrieve.

Query parameters

include array **Optional**

Additional fields to include in the response. See the `include` parameter for [listing Conversation items above](#) for more information.

Returns

Returns a [Conversation Item](#).

Example request

javascript

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const item = await client.conversations.items.retrieve(
5   "conv_123",
6   "msg_abc"
7 );
8 console.log(item);
```

Response

```
1 {
2   "type": "message",
3   "id": "msg_abc",
4   "status": "completed",
5   "role": "user",
6   "content": [
7     {"type": "input_text", "text": "Hello!"}
8   ]
9 }
```

Delete an item

```
DELETE https://api.openai.com/v1/conversations/{conversation_id}/items/{item_id}
```

Delete an item from a conversation with the given IDs.

Path parameters

conversation_id string **Required**

The ID of the conversation that contains the item.

item_id string **Required**

The ID of the item to delete.

Returns

Returns the updated [Conversation](#) object

Returns the updated `Conversation` object.

Example request

javascript

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const conversation = await client.conversations.items.delete(
5   "conv_123",
6   "msg_abc"
7 );
8 console.log(conversation);
```

Response

```
1 {
2   "id": "conv_123",
3   "object": "conversation",
4   "created_at": 1741900000,
5   "metadata": {"topic": "demo"}
6 }
```

The conversation object

created_at integer

The time at which the conversation was created, measured in seconds since the Unix epoch.

id string

The unique ID of the conversation.

metadata

Set of 16 key-value pairs that can be attached to an object. This can be useful for storing additional information about the object in a structured format, and querying for objects via API or the dashboard. Keys are strings with a maximum length of 64 characters. Values are strings with a maximum length of 512 characters.

object string

The object type, which is always `conversation` .

The item list

A list of Conversation items.

data array

A list of conversation items.

Show possible types

first_id string

The ID of the first item in the list.

has_more boolean

Whether there are more items available.

last_id string

The ID of the last item in the list.

object string

The type of object returned, must be `list` .

PREVIOUS

Responses

NEXT

Streaming events